

Plan de Présentation - SPRING_CommerceFlow-MS

Durée : 15 minutes | Présentateurs : Ayoub & Ayman

📄 Structure Globale

Section	Durée	Présentateur	Slides
Introduction	1 min	Ayoub	1-2
Contexte & Problématique	2 min	Ayoub	3-4
Architecture & Conception	4 min	Ayman	5-8
Implémentation Technique	4 min	Ayoub	9-12
Démo Live	3 min	Ayman	-
Conclusion & Questions	1 min	Ayoub	13

Total : 15 minutes

🎯 Partie 1 : Introduction (1 min) - **AYOUB**

Slide 1 : Page de Garde

- Titre du projet
- Vos noms
- Encadrant

Slide 2 : Agenda

Script :

"Bonjour, nous allons vous présenter SPRING_CommerceFlow-MS, une plateforme e-commerce basée sur une architecture microservices. Notre présentation se déroulera en 4 parties : le contexte, l'architecture, l'implémentation technique, et une démonstration live."

🔍 Partie 2 : Contexte & Problématique (2 min) - **AYOUB**

Slide 3 : Problématique

Contenu :

- Limites des architectures monolithiques
- Tableau comparatif :
 - ❌ Couplage fort
 - ❌ Scalabilité limitée
 - ❌ Déploiement risqué
 - ❌ Technologie unique

Script :

"Les applications monolithiques présentent des limitations critiques : un couplage fort qui nécessite le redéploiement complet pour chaque modification, une scalabilité limitée car on ne peut pas scaler un seul composant indépendamment, et un déploiement risqué où un bug affecte tout le système."

Slide 4 : Objectifs du Projet

Contenu :

- ☒ Architecture microservices scalable
- ☒ Sécurité OAuth2/OIDC avec Keycloak
- ☒ Service Discovery avec Eureka
- ☒ Résilience avec Circuit Breaker
- ☒ Persistance polyglotte (MongoDB + MySQL)

Script :

Partie 3 : Architecture & Conception (4 min) - **AYMAN**

Slide 5 : Architecture Globale

Contenu :

- Diagramme d'architecture complet
- Composants :
 - API Gateway (Port 9000)
 - Eureka Server (Port 8761)
 - Product Service (Port 8080) → MongoDB
 - Order Service (Port 8081) → MySQL
 - Inventory Service (Port 8082) → MySQL
 - Keycloak (Port 8181)

Script :

"Notre architecture se compose de 4 microservices métier : Product Service qui gère le catalogue avec MongoDB pour sa flexibilité, Order Service qui traite les commandes avec MySQL pour la conformité ACID, et Inventory Service qui gère les stocks. Tous ces services communiquent via une API Gateway centralisée."

Slide 6 : Service Discovery & Load Balancing

Contenu :

- Schéma Eureka
- Flux de résolution :
 1. Client → Gateway
 2. Gateway → ServiceResolver
 3. LoadBalancerClient → Eureka Cache
 4. Sélection Round-Robin
 5. Requête vers instance choisie

Script :

"Nous utilisons Eureka pour le service discovery. Chaque service s'enregistre automatiquement auprès d'Eureka. Le Gateway utilise un ServiceResolver custom qui interroge le LoadBalancerClient pour sélectionner dynamiquement une instance disponible via l'algorithme Round-Robin."

Slide 7 : Sécurité OAuth2 avec Keycloak

Contenu :

- Flux d'authentification
- Types de tokens (Access Token vs Refresh Token)
- Vérification JWT au niveau du Gateway

Script :

"La sécurité est centralisée avec Keycloak. L'utilisateur s'authentifie une fois et reçoit un JWT. Le Gateway valide ce token pour chaque requête avant de le transmettre aux microservices. Nous utilisons des Access Tokens de courte durée (5 min) et des Refresh Tokens pour renouveler l'accès."

Slide 8 : Résilience avec Resilience4j

Contenu :

- Machine à états du Circuit Breaker
- États : CLOSED → OPEN → HALF_OPEN
- TimeLimiter (3s timeout)
- Retry Pattern (3 tentatives)

Script :

"Pour garantir la résilience, nous avons implémenté le pattern Circuit Breaker. Si 50% des 10 derniers appels échouent, le circuit s'ouvre et rejette immédiatement les requêtes pendant 5 secondes. Ensuite, il passe en état HALF_OPEN pour tester 3 requêtes avant de décider de fermer ou rouvrir le circuit."

Partie 4 : Implémentation Technique (4 min) - **AYOUB**

Slide 9 : Stack Technologique

Contenu :

```
Backend:
- Spring Boot 4.0.0
- Spring Cloud 2025.1.0
- Java 21

Bases de données:
- MongoDB (Product Service)
- MySQL 8.x (Order & Inventory)

Infrastructure:
- Eureka (Service Discovery)
- Keycloak (IAM)
- Resilience4j (Circuit Breaker)
- Docker (Conteneurisation)
```

Script :

"Notre stack technique repose sur Spring Boot 4 et Java 21. Nous utilisons une persistance polyglotte : MongoDB pour le catalogue produits grâce à sa flexibilité, et MySQL pour les commandes et l'inventaire pour garantir la conformité ACID des transactions."

Slide 10 : Communication Inter-Services

Contenu :

- Code OpenFeign :

```
@FeignClient(name = "inventory-service")
public interface InventoryClient {
    @GetMapping("/api/inventory/{skuCode}")
    InventoryResponse checkInventory(@PathVariable String skuCode);
}
```

Script :

"La communication entre services utilise OpenFeign, un client HTTP déclaratif. Par exemple, l'Order Service interroge l'Inventory Service pour vérifier la disponibilité avant de créer une commande. Cette approche est type-safe et facilement testable avec WireMock."

Slide 11 : ServiceResolver & MvcUtils

Contenu :

```
public URI resolve(String serviceName, String path) {
    ServiceInstance instance = loadBalancerClient.choose(serviceName);
    return URI.create(instance.getUri().toString() + path);
}

// Dans Routes.java
MvcUtils.setRequestUrl(newRequest, resolvedUri);
HandlerFunctions.http().handle(newRequest);
```

Script :

"Dans Gateway MVC, nous avons créé un ServiceResolver custom qui utilise LoadBalancerClient pour résoudre les noms de services en URIs physiques. MvcUtils.setRequestUrl modifie l'URL de destination avant de forwarder la requête vers l'instance sélectionnée."

Slide 12 : Tests & Qualité

Contenu :

- Pyramide des tests :
 - Tests Unitaires (Mockito) - 100%
 - Tests d'Intégration (MockMvc) - 70%

- Tests E2E (RestAssured)
- Tests WireMock (Feign Clients)
- Couverture : > 70%

Script :

"Nous avons adopté une stratégie de tests complète : tests unitaires avec Mockito pour la logique métier, tests d'intégration avec MockMvc pour valider les endpoints, et WireMock pour simuler les services externes. Notre couverture de code dépasse 70%."

Partie 5 : Démo Live (3 min) - **AYMAN**

Préparation avant la présentation :

- ☒ Tous les services démarrés (Eureka, Keycloak, Gateway, 3 microservices)
- ☒ Frontend prêt
- ☒ Données de test chargées
- ☒ Postman/Frontend ouvert

Scénario de Démo :

1. Dashboard Eureka (30 sec)

Action : Ouvrir <http://localhost:8761>

Montrer :

- Les 3 services enregistrés (product-service, order-service, inventory-service)
- Statut UP de chaque instance

Script :

"Voici le dashboard Eureka. On voit nos 3 microservices enregistrés et disponibles. Si on scale un service, il apparaîtra automatiquement ici."

2. Authentification Keycloak (30 sec)

Action : Se connecter via le frontend

Montrer :

- Page de login Keycloak
- Redirection après authentification
- Token JWT dans le localStorage (DevTools)

Script :

"L'utilisateur se connecte via Keycloak. Une fois authentifié, il reçoit un JWT qui sera utilisé pour toutes les requêtes suivantes."

3. Flux Complet E-Commerce (1 min 30)

Action : Parcours utilisateur complet

Étapes :

1. **Consulter le catalogue** (GET /api/product)
 - Afficher la liste des produits
2. **Vérifier le stock** (GET /api/inventory/{sku})
 - Montrer la quantité disponible
3. **Passer une commande** (POST /api/order)
 - Créer une commande
 - Montrer la validation automatique du stock
4. **Consulter la commande** (GET /api/order/{id})
 - Afficher les détails de la commande créée

Script :

"Je vais maintenant passer une commande. Le système consulte automatiquement l'inventaire via OpenFeign. Si le stock est suffisant, la commande est créée et le stock est décrémenté. Toutes ces opérations passent par le Gateway qui route vers les bons services grâce à Eureka."

4. Circuit Breaker en Action (30 sec)

Action : Arrêter l'Inventory Service

Montrer :

- Tentative de commande

- Circuit Breaker s'ouvre
- Message de fallback
- Actuator health endpoint (circuit OPEN)

Script :

"Si je stoppe l'Inventory Service et tente une commande, le Circuit Breaker détecte les échecs et ouvre le circuit après 5 échecs. Les requêtes suivantes sont immédiatement rejetées avec un message de fallback, protégeant ainsi le système."

📖 Partie 6 : Conclusion (1 min) - **AYOUB**

Slide 13 : Résultats & Perspectives

Réalisations :

- ☒ Architecture microservices complète (78% de progression)
- ☒ 4 services métier fonctionnels
- ☒ Sécurité OAuth2/OIDC opérationnelle
- ☒ Service Discovery & Load Balancing
- ☒ Mécanismes de résilience (Circuit Breaker, Retry, TimeLimiter)
- ☒ Tests automatisés (> 70% coverage)

Perspectives :

- ☐ Notification Service avec Kafka
- ☐ Frontend React complet
- ☐ Pipeline CI/CD
- ☐ Déploiement Kubernetes
- ☐ Distributed Tracing (Zipkin)

Script :

"En conclusion, nous avons développé une plateforme e-commerce moderne basée sur une architecture microservices robuste. Le projet est à 78% de complétion avec tous les composants principaux fonctionnels. Les prochaines étapes incluent le Notification Service avec Kafka, la finalisation du frontend, et le déploiement sur Kubernetes. Merci pour votre attention, nous sommes prêts pour vos questions."

📝 Conseils pour la Présentation

Pour Ayoub :

1. **Introduction** : Parler clairement et lentement, établir le contact visuel
2. **Contexte** : Utiliser des exemples concrets (Netflix, Amazon) pour illustrer les problèmes monolithiques
3. **Implémentation** : Montrer du code simple et commenté, éviter les détails trop techniques
4. **Conclusion** : Être enthousiaste sur les perspectives

Pour Ayman :

1. **Architecture** : Pointer les éléments du diagramme au fur et à mesure
2. **Service Discovery** : Expliquer avec un exemple concret (scaling de 1 à 5 instances)
3. **Sécurité** : Simplifier le flux OAuth2, utiliser des analogies (badge d'accès)
4. **Démo** : Tester TOUT avant, avoir un plan B si problème technique

Timing :

- 🕒 Répéter plusieurs fois pour respecter les 15 minutes
- 🕒 Prévoir 30 secondes de marge pour les transitions
- 🕒 Si en retard, sauter les détails techniques, garder la démo

Questions Probables :

1. **Pourquoi MongoDB pour Product et MySQL pour Order ?**
→ Flexibilité du schéma vs conformité ACID
2. **Comment gérez-vous les transactions distribuées ?**
→ Pattern Saga (à implémenter) ou compensation manuelle
3. **Pourquoi Gateway MVC et pas Reactive ?**
→ Cohérence avec nos services blocking, simplicité de debugging

4. Comment testez-vous la communication inter-services ?

→ WireMock pour simuler les APIs externes

5. Quelle est la stratégie de déploiement ?

→ Docker actuellement, Kubernetes prévu (Blue-Green deployment)

Recommandations Slides

Design :

- Utiliser le template EMSI avec logos
- Maximum 5-6 points par slide
- Diagrammes clairs et colorés
- Code avec syntax highlighting
- Animations subtiles (apparition progressive)

Contenu :

- Titre clair sur chaque slide
 - Numérotation des slides
 - Icônes pour illustrer ( sécurité,  résilience, etc.)
 - Captures d'écran de l'architecture
 - GIFs pour montrer les flux (optionnel)
-

Checklist Avant Présentation

Technique :

- ☐ Tous les services démarrés et testés
- ☐ Frontend fonctionnel
- ☐ Données de test chargées
- ☐ Postman configuré (backup si frontend fail)
- ☐ Eureka dashboard accessible
- ☐ Actuator endpoints testés
- ☐ Batterie chargée (laptop)
- ☐ Adaptateur HDMI/VGA disponible

Présentation :

- ☐ Slides finalisées et exportées en PDF (backup)
- ☐ Répétition complète chronométrée
- ☐ Répartition claire Ayoub/Ayman
- ☐ Notes de présentation imprimées
- ☐ Questions/réponses préparées
- ☐ Tenue professionnelle

Démo :

- ☐ Scénario testé 3 fois minimum
 - ☐ Plan B si service crash
 - ☐ Captures d'écran de backup
 - ☐ Vidéo de démo (dernier recours)
-

Bonne chance pour votre présentation ! 